

WORLD WISE PLATFORM

Research & Development Documentation

Platform Evolution: Mobile Racing Application → WebGL Educational Ecosystem **R&D Period:** July 2025 – March 2026

1. Executive Summary

World Wise originally commenced as a mobile-first Unity racing application intended for iOS and Android distribution.

During development, strategic and technical requirements led to a platform pivot toward a WebGL-first browser deployment model. This shift was driven by:

- School device restrictions preventing native app installations
- Requirement for browser-based access (no App Store deployment)
- Need for student authentication and session control
- Integration with a Blazor-based educational portal
- Cross-device compatibility (desktop, tablet, Chromebook, mobile browser)
- Centralised analytics and progress tracking

As a result, three parallel development tracks emerged:

Track A – Racing Game: Originally architected as a mobile app, now undergoing WebGL migration and performance re-engineering.

Track B – Math Game: Designed and built WebGL-native from inception, specifically for browser deployment in educational environments.

Track C – Browser-Native Games: Educational games and interactive quiz templates built using HTML5 technologies (Phaser.js, React, Vanilla JavaScript), requiring solutions for cross-browser compatibility, responsive scaling, and Blazor portal integration without Unity's interop layer. A centralised games portal was developed to provide unified access across all game technologies. Additionally, 17+ interactive quiz question templates were developed as Blazor-ready prototypes, requiring custom solutions for mobile touch handling, floating physics animations, circular angle mathematics, and SVG-based line drawing.

The transition introduced significant technical uncertainty due to Unity WebGL limitations, browser GPU constraints, memory ceilings, HTML5 cross-browser inconsistencies, and lack of official guidance for advanced WebGL and browser-native implementations.

The following sections document the experimental development undertaken to resolve these uncertainties.

2. Track A – Racing Game (Mobile App → WebGL Migration)

2.1 Background

The racing game was initially developed as a native Unity mobile application. Performance assumptions were based on mobile GPU capabilities and native memory access.

Migration to WebGL introduced strict constraints:

- Browser GPU sandbox limitations

- WebGL 2.0 rendering restrictions
- Memory heap limits (2GB max; iOS Safari ~500MB practical ceiling)
- Removal of native plugin functionality
- Increased draw call sensitivity

This required systematic performance re-engineering.

2.2 WebGL Performance Optimisation – Procedural City Environment (Jul 2025 – Feb 2026)

Technical Problem

CityGen3D procedural city generation produced scenes containing 700,000–800,000 triangles.

WebGL practical rendering ceiling for stable browser gameplay is approximately 100,000–150,000 triangles.

Initial WebGL builds resulted in:

- Frame rates as low as 12fps
- Memory instability
- Browser crashes on lower-tier devices

Technical Uncertainty

It was unknown whether:

- The procedural generation system could be reduced below WebGL thresholds
- Hidden geometry was contributing to performance degradation
- WebGL memory limits would prevent viable city-scale environments
- The asset's LOD systems were sufficient for browser deployment

No WebGL optimisation guidance was provided by the asset vendor.

Experimental Process

- Systematic tri-count auditing by disabling object categories
- Identified hidden duplicate buildings below terrain (~300k tris)
- Enabled "Batched Low Poly" LOD mode
- Reduced roadside spawn density (~300k tris removed)
- Disabled URP environment elements (~240k tris removed)
- Disabled real-time shadows for WebGL builds
- Reconfigured Generator settings specifically for WebGL target

Quantified Outcome

- Reduced scene from **~780,000 triangles** → **~95,000 triangles**
- Frame rate improved from **~12fps** → **55–60fps** on mid-tier browser devices
- Stabilised WebGL memory usage below crash thresholds

Evidence

- CityGen3D-Test project
 - Generator configuration files
 - Scene tri-count profiling logs
 - Performance benchmarking records
-

2.3 WebGL Build Size Optimisation (Aug–Sept 2025)

Problem

Initial builds exceeded optimal browser download and memory thresholds.

Experimental Measures

- Enabled Brotli compression
- Managed code stripping level optimisation
- Removed unused shaders and materials
- Texture compression reconfiguration
- Asset bundle separation strategy

Result

- Reduced build size
 - Improved load times
 - Lower memory overhead in browser environment
-

2.4 Unity WebGL Module Compatibility Research (Jul–Aug 2025)

- Installed WebGL build modules via Unity Hub
 - Evaluated platform-specific build settings
 - Tuned memory allocation settings
 - Tested cross-browser compatibility (Chrome, Safari, Edge)
-

3. Track B – Educational Math Game (WebGL-Native)

The math game was architected WebGL-first and required solving browser-specific limitations from inception.

3.1 Mobile WebGL Video Playback System (Sept–Oct 2025)

Technical Problem

Unity's VideoPlayer component fails in mobile WebGL builds. Issues included:

- Video freeze
- Audio desynchronisation
- Failure to autoplay
- Browser security blocking playback

Unity provides no official solution for mobile WebGL video support.

Technical Uncertainty

It was unknown whether:

- HTML5 video overlays could integrate with WebGL canvas safely
- Input control would remain functional
- Audio synchronisation could be maintained
- Responsive scaling across device sizes was achievable

Experimental Solution

- Custom `.jslib` JavaScript plugin injecting HTML5 `<video>` element
- C# ↔ JavaScript bridge using `[DllImport("__Internal")]`
- Responsive layout logic for phone/tablet/desktop
- Input locking while video active
- Auto-fit tutorial text algorithm
- Minimize/expand UI system

Outcome

- Stable playback on iOS Safari and Android Chrome
- No canvas freezes
- Fully responsive scaling

Evidence

- TutorialBubble.jslib
- BubbleTest project
- Git commit logs

3.2 Equation-Based Match-3 Game Mechanic (Aug–Sept 2025)

Technical Problem

Traditional match-3 engines detect color alignment only.

World Wise required detection of valid mathematical equations, such as:

$3 + 5 = 8$

No framework existed for equation-based tile validation.

Technical Uncertainty

It was unknown whether:

- Real-time equation validation could coexist with gravity mechanics
- Operator precedence logic could function within grid constraints
- Diagonal falling resolution would remain stable

Experimental Development

- Rebuilt CombineManager.cs
- Implemented operator precedence evaluation
- Numbers (0–99) fall dynamically
- Operators remain static
- Diagonal gravity resolution
- Custom level editor

Outcome

- Fully functional equation-solving mechanic
- Stable falling behaviour
- Scalable to expanded number ranges

3.3 Two-Digit Number Expansion (0–99) (Sept 2025)

Problem

Original template supported only digits 1–9.

Educational requirements required 0–99.

Experimental Changes

- Remapped ID system (0–99 numbers, 100–104 operators)
 - Generated 105 sprites per difficulty level
 - Editor UI updates
 - Backward compatibility preserved
-

3.4 Blazor ↔ Unity WebGL Portal Integration (Nov–Dec 2025)

Technical Problem

Unity WebGL has limited interop capabilities.

The game required:

- Student login authentication
- Session token passing
- Auto-save functionality
- Portal-controlled progress tracking

Technical Uncertainty

It was unclear whether:

- Secure URL parameter parsing could be achieved
- Bidirectional messaging would remain stable
- Session persistence would survive page reloads

Experimental Solution

- BlazorUnityBridge.cs messaging layer
- URL parameter parsing (studentId, sessionId, token)
- Pull-based portal state queries
- Auto-save every 60 seconds and on level completion

Outcome

- Stable session management
 - Secure student progress tracking
 - Portal-controlled integration
-

3.5 LeanTween State Corruption Debugging (Sept 2025)

Problem

Random freeze during scene loading in Unity Editor.

Experimental Investigation

- Traced issue to LeanTween static state persistence
- Implemented `LeanTween.reset()` before transitions

- Documented reproduction conditions

Result

- Eliminated freeze
 - Stabilised development workflow
-

3.6 Educational Compliance Refactor (Aug 2025)

Problem

Commercial template contained:

- Lives system
- Ads
- Bonus spin wheel
- Boosters

These were unsuitable for school environments.

Experimental Process

- Isolated monetisation systems
- Disabled ad calls
- Converted lives to unlimited play
- Removed gambling-style reward loops

Result

- School-safe gameplay
 - Stable system state without monetisation dependencies
-

3.7 Infant Curriculum Variant (Foundation–Year 1) (Oct 2025)

Adjustments

- Restricted numbers to 0–10
 - Removed $\times$ and $\div$ operators
 - Built 80 progressive levels
 - Editor restrictions preventing invalid equations
-

4. Track C – Browser-Native Educational Games (HTML5/JavaScript)

In parallel with Unity WebGL development, a suite of browser-native games was developed using HTML5 technologies (Phaser.js, React, Vanilla JavaScript). These required solving cross-browser compatibility, responsive scaling, and portal integration challenges distinct from Unity.

4.1 Kangaroo Hop – Phaser.js Responsive Scaling & Asset Loading (Sept 2025)

Technical Problem

The Kangaroo Hop arcade game was built using Phaser.js with fixed 800x600 canvas dimensions. School deployment required responsive scaling across diverse device types (Chromebooks, tablets, mobile browsers, desktop).

Additionally, GitHub Pages deployment introduced asset loading failures where parallax background images returned 404 errors despite functioning in local development.

Technical Uncertainty

It was unknown whether:

- Phaser's scale manager could dynamically handle viewport changes without visual distortion
- Parallax scrolling backgrounds would maintain visual integrity at non-native aspect ratios
- GitHub Pages asset pathing conventions differed from local development environments
- Touch input mapping would function consistently across mobile browsers

Experimental Process

- Investigated GitHub Pages asset resolution behaviour
- Identified case-sensitivity and folder structure requirements for static hosting
- Relocated images from `parallax/` to `assets/images/outback-background/`
- Tested responsive scaling across Chrome, Safari, Edge on multiple device sizes
- Validated touch input on iOS Safari and Android Chrome

Outcome

- Resolved asset loading failures on GitHub Pages
- Achieved consistent visual presentation across device sizes
- Confirmed touch input compatibility

Evidence

- my-kangaroo-game repository
- Git commit history documenting background loading fix
- GitHub Pages deployment logs

4.2 Wordle – React Teacher Customisation Architecture (Sept–Oct 2025)

Technical Problem

Standard Wordle implementations use a fixed daily word list. Educational deployment required teacher-assigned custom word lists to align with curriculum vocabulary.

This introduced requirements for:

- URL parameter injection of custom words
- Validation of teacher-provided words against game rules
- Mode switching between daily words and teacher-assigned words

Technical Uncertainty

It was unknown whether:

- URL parameter parsing could securely accept variable-length word arrays
- Teacher-provided words could be validated in real-time against dictionary and length rules
- Mode selection logic could coexist without state conflicts
- Custom word lists could persist across sessions without backend infrastructure

Experimental Development

- Designed URL parameter schema for word list injection
- Implemented client-side word validation (5-letter requirement, character validation)

- Developed mode selection architecture (daily vs teacher-assigned)
- Created fallback behaviour for invalid word submissions

Outcome

- Functional URL parameter parsing for custom words
- Client-side validation preventing invalid game states
- Architecture prepared for portal integration

Evidence

- wordle-game repository
 - URL parameter handling implementation
 - Git commit history
-

4.3 Memory Mania – Educational Theme System (Oct 2025)

Technical Problem

Memory matching games typically use generic imagery. Educational deployment required themed content appropriate for Australian school curriculum, including Australian animals and educational categories.

Technical Uncertainty

It was unknown whether:

- Multiple theme systems could coexist without increasing load times
- Image assets could be efficiently loaded on demand per theme selection
- Cognitive difficulty scaling could be implemented via grid size variation

Experimental Development

- Implemented theme selection system (Australian animals, shapes, numbers)
- Developed progressive difficulty via grid size (4x3, 4x4, 5x4, 6x5)
- Optimised image preloading to minimise theme switch latency

Outcome

- Multiple educational themes functional
- Progressive difficulty scaling operational
- Theme switching without performance degradation

Evidence

- memory-game repository
 - Theme configuration files
-

4.4 Spelling Snake – Word-Based Arcade Mechanic (Oct 2025)

Technical Problem

Traditional snake games collect generic items for points. Educational requirements mandated letter collection forming valid spelling words, combining arcade mechanics with literacy practice.

Technical Uncertainty

It was unknown whether:

- Real-time word validation could function within arcade timing constraints
- Letter spawning algorithms could ensure valid words remained achievable
- Dictionary lookup performance would impact frame rate

Experimental Development

- Implemented real-time letter collection tracking
- Developed word validation against curriculum-appropriate dictionary
- Created letter spawning algorithm ensuring word completion possibility
- Balanced arcade pacing with educational letter distribution

Outcome

- Functional spelling-based collection mechanic
- Real-time validation without performance impact
- Educational word practice within arcade format

Evidence

- spelling-snake-game repository
- Word validation implementation

4.5 2048 – Vanilla JavaScript Portal Integration (Oct–Nov 2025)

Technical Problem

The 2048 puzzle game required integration with the educational portal while maintaining minimal codebase complexity. Vanilla JavaScript implementation needed URL parameter handling and session management without framework dependencies.

Technical Uncertainty

It was unknown whether:

- Vanilla JavaScript could implement secure URL parameter parsing matching React/Unity implementations
- High score persistence could function via localStorage with portal session context
- Return-to-portal navigation could be implemented without framework routing

Experimental Development

- Implemented native JavaScript URL parameter parsing
- Developed localStorage high score system with session context
- Created portal navigation integration

Outcome

- Consistent URL parameter handling across technology stack
- Session-aware high score persistence
- Portal integration without framework overhead

Evidence

- 2048-game repository
- URL parameter implementation

4.6 Quiz Templates – Interactive Question Prototypes for Blazor Integration (Feb–Mar 2026)

A suite of 17+ interactive quiz question templates was developed as HTML5/CSS/JavaScript prototypes. These templates demonstrate interaction patterns for conversion to Blazor components, requiring solutions to cross-browser touch handling, animation performance, and complex user input validation.

4.6.1 Draggable Clock – Circular Angle Mathematics (Feb 2026)

Technical Problem

The draggable clock component required students to rotate clock hands to set times. Standard angle calculations fail when rotation crosses the $0^\circ/360^\circ$ boundary (e.g., dragging from 11 o'clock to 1 o'clock produces a -330° delta instead of $+60^\circ$).

Additionally, minute hand rotation needed to synchronise the hour hand position (12 full minute rotations = 1 hour hand rotation), requiring continuous rotation tracking across multiple revolutions.

Technical Uncertainty

It was unknown whether:

- Angle wrap-around at $0^\circ/360^\circ$ could be resolved without visual jumping
- Hour hand synchronisation could track cumulative minute hand rotations
- Touch input precision would be sufficient for clock hand selection on mobile devices
- The mathematical model would handle both clockwise and counter-clockwise rotation

Experimental Development

- Implemented boundary detection with delta normalisation:
 - If delta > 180° , subtract 360° (counter-clockwise wrap)
 - If delta < -180° , add 360° (clockwise wrap)
- Created `totalMinuteRotation` accumulator tracking $0-4320^\circ$ (12 full rotations)
- Developed hour hand position derived from cumulative minute rotation
- Tested touch target sizes across iOS Safari and Android Chrome

Outcome

- Smooth rotation across 12 o'clock boundary in both directions
- Accurate hour hand synchronisation with minute hand
- Functional touch interaction on mobile devices

Evidence

- quiz-templates repository: `js/clock.js` lines 205-217 (wrap-around logic)
 - `js/clock.js` lines 267-275 (angle conversion mathematics)
 - `demo-clock.html`
-

4.6.2 Mobile Touch Drag-Drop with Auto-Scroll (Feb 2026)

Technical Problem

HTML5 native drag-and-drop API does not function reliably on mobile browsers. Touch-based dragging was required for sorting activities (e.g., dragging shapes into category jars).

Additionally, when drop targets existed below the visible viewport, standard touch handling provided no mechanism for auto-scrolling while dragging.

Technical Uncertainty

It was unknown whether:

- Touch events could reliably replace drag events across iOS Safari, Android Chrome, and Samsung Internet
- Auto-scroll during touch-drag could be implemented without interfering with drop detection
- Visual drag feedback (ghost element) could be positioned correctly during touch movement
- `elementFromPoint()` would correctly identify drop targets beneath the dragged element

Experimental Development

- Replaced `dragstart/dragend` with `touchstart/touchmove/touchend` handlers
- Implemented `{ passive: false }` to prevent browser scroll interference
- Created clone-based visual feedback appended to `document.body`
- Developed auto-scroll system detecting proximity to viewport edges:
 - `SCROLL_EDGE_SIZE` : 60px trigger zone
 - `SCROLL_SPEED` : 8px per frame
- Used `elementFromPoint()` with drag ghost temporarily hidden for accurate hit detection

Outcome

- Consistent drag-drop functionality across mobile browsers
- Auto-scroll enabling drops below initial viewport
- Accurate drop zone detection

Evidence

- quiz-templates repository: `js/drag-drop.js` lines 39-51 (auto-scroll config)
- `js/drag-drop.js` lines 198-302 (touch event handling)
- `demo-drag-drop.html`

4.6.3 Balloon Pop – Floating Physics Animation Engine (Mar 2026)

Technical Problem

The balloon pop phonics game required multiple balloons floating with realistic physics:

- Natural bouncing off viewport boundaries
- Smooth velocity changes (not linear direction reversals)
- Handling dynamic container resizes
- Maintaining 60fps performance while animating 4-6 elements
- Preventing balloon overlap on initialisation

Technical Uncertainty

It was unknown whether:

- Custom physics simulation could achieve natural-looking movement without a physics library
- Velocity easing would produce visually acceptable bounce behaviour
- `requestAnimationFrame` performance would remain stable during DOM mutations (popping balloons)
- Initial positioning could prevent balloon overlap without complex collision detection

Experimental Development

- Implemented velocity smoothing with easing constant (0.02 per frame)
- Created bounce physics with randomised intensity on boundary collision
- Developed grid-based initial positioning to prevent overlap
- Cached balloon dimensions to avoid layout thrashing
- Implemented dynamic container dimension recalculation during animation

Quantified Parameters

- Balloon body height ratio: **65%** of total SVG height (experimentally determined)
- Velocity easing: `state.vx += (state.targetVx - state.vx) * 0.02`
- Bounce randomisation: `1 + Math.random() * 2` velocity units

Outcome

- Natural floating animation at 60fps
- Responsive to viewport resizes
- No balloon overlap on game start
- Stable performance during balloon removal

Evidence

- quiz-templates repository: `demo-balloon-pop.html` lines 1260-1402 (physics engine)
 - `demo-balloon-pop.html` lines 1296-1308 (grid positioning)
-

4.6.4 SVG Line Matching with Bézier Curves (Feb 2026)

Technical Problem

The line matching game required drawing smooth connecting lines between left and right columns. Students click connection points to draw lines linking related items (e.g., shapes to their number of sides).

Lines needed to:

- Render smoothly using SVG paths
- Update in real-time during drag operations
- Convert screen coordinates to SVG coordinate space
- Use curved paths for visual appeal (not straight lines)

Technical Uncertainty

It was unknown whether:

- Bézier curve control points could produce consistently appealing curves across varying distances
- Coordinate system translation from viewport to SVG container would remain accurate during scroll
- Real-time path updates would perform adequately during touch/mouse movement
- Hit detection for line removal would function on curved paths

Experimental Development

- Implemented cubic Bézier curves with symmetric control points at horizontal midpoint
- Created coordinate translation: screen position → container-relative → SVG coordinates
- Developed real-time path generation during drag operations
- Used dashed stroke animation for visual feedback

Bézier Formula

- Control points placed at `(midX, y1)` and `(midX, y2)`
- Path: `M x1 y1 C midX y1, midX y2, x2 y2`

Outcome

- Smooth curved connections between match pairs
- Accurate coordinate translation across viewport positions
- Real-time drag visualisation

Evidence

- quiz-templates repository: `demo-line-match.html` lines 857-900 (Bézier implementation)
 - `demo-line-match.html` lines 889-895 (coordinate translation)
-

4.6.5 CSS Pointer Events Layering for Complex Interactions (Feb 2026)

Technical Problem

Interactive elements required selective click-through behaviour:

- Balloon containers needed to be click-through (transparent to events)
- Only the balloon body (SVG ellipse) should capture clicks
- Popped/disabled balloons should not capture events
- String elements should not interfere with balloon clicking

Standard CSS positioning does not solve event propagation for overlapping elements.

Technical Uncertainty

It was unknown whether:

- `pointer-events` CSS property would function consistently across browsers
- Nested pointer-events inheritance could be overridden at child level
- State-based pointer-events changes would update without requiring element recreation

Experimental Development

- Applied `pointer-events: none` to parent container
- Applied `pointer-events: auto` to interactive child (SVG ellipse only)
- Implemented state-based pointer-events for disabled/popped states
- Tested across Chrome, Safari, Firefox, Edge

Outcome

- Precise click targeting on balloon bodies only
- String and container elements correctly ignored
- State changes correctly disable interaction

Evidence

- quiz-templates repository: `demo-balloon-pop.html` lines 50, 76, 209-211 (CSS implementation)
-

4.6.6 Progress Bar with Animated Spinning Compass (Mar 2026)

Technical Problem

A progress indicator was required across all 17+ templates. Design requirements included:

- Gradient-filled progress bar

- Animated compass icon following progress position
- Compass spins faster when progressing forward
- Compass spins backwards when returning to previous question
- Continuous idle spin when stationary

Technical Uncertainty

It was unknown whether:

- `requestAnimationFrame`-based rotation could smoothly transition between speed states
- CSS transitions on position could synchronise with JavaScript rotation updates
- The animation would perform adequately across all 17 templates simultaneously
- Backward spin direction could be achieved by negating rotation speed

Experimental Development

- Implemented continuous rotation loop using `requestAnimationFrame`
- Created speed state system: `slowSpeed` (120°/s idle), `fastSpeed` (720°/s active)
- Developed direction control via signed speed values (negative = backwards)
- Used `setTimeout` to return to idle speed after 500ms
- Applied CSS transition on compass position for smooth following

Outcome

- Smooth continuous rotation at 60fps
- Responsive speed changes on progress updates
- Backward spin on "previous question" actions
- Consistent implementation across all templates

Evidence

- quiz-templates repository: `demo-progress-bar.html` (complete implementation)
- `DEVELOPER-GUIDE.md` lines 771-883 (documentation)

5. Cross-Platform Portal Integration Architecture

5.1 Unified Integration Schema (Nov–Dec 2025)

Technical Problem

The educational platform comprised games built across five distinct technologies:

- Unity WebGL (Math Crush, Racing Game)
- Phaser.js (Kangaroo Hop)
- React (Wordle)
- Vanilla JavaScript (2048)
- HTML5 Canvas (Memory Mania, Spelling Snake)

Each technology handled URL parameters, session management, and portal communication differently. A unified integration architecture was required for Victor's Blazor portal.

Technical Uncertainty

It was unknown whether:

- A consistent URL parameter schema could function across all frameworks
- Session token handling could be standardised despite framework differences

- Portal navigation ("Return to Portal") could be implemented consistently
- Progress tracking APIs could accept data from heterogeneous game technologies

Experimental Solution

- Designed standardised URL parameter schema: `?studentId=X&sessionId=Y&token=Z`
- Documented framework-specific parameter parsing implementations
- Created consistent `postMessage` API pattern for portal communication
- Established "Return to Portal" navigation standard across all games

Outcome

- Unified integration interface regardless of underlying technology
- Consistent student session handling
- Portal-agnostic game architecture enabling future game additions

Evidence

- sasco-games-portal repository
- Individual game URL parameter implementations
- Integration documentation

5.2 Games Portal Hub Development (Oct–Nov 2025)

Technical Problem

Multiple games required a centralised access point suitable for school environments. The portal needed to:

- Present games with clear educational vs recreational categorisation
- Function across all school device types
- Load efficiently on limited bandwidth connections
- Maintain visual appeal for student engagement

Technical Uncertainty

It was unknown whether:

- Pure HTML/CSS implementation could achieve required interactivity without JavaScript
- Responsive grid layouts would function across Chromebook, tablet, and mobile viewports
- CSS animations would perform adequately on lower-tier school devices

Experimental Development

- Implemented CSS-only animation system (floating shapes, card hover effects)
- Developed responsive grid using CSS `auto-fill` with minimum column widths
- Created tag-based categorisation system (Educational, Fun, Spelling, Maths, etc.)
- Optimised for minimal payload and fast load times

Outcome

- Sub-second load times on standard connections
- Consistent presentation across device types
- No JavaScript dependencies for core functionality

Evidence

- sasco-games-portal repository
- CSS animation implementation

- Responsive breakpoint configuration

6. Strategic Outcome

The transition from a mobile racing application to a WebGL-first educational platform required:

- Re-engineering for browser GPU constraints
- Solving undocumented Unity WebGL limitations
- Creating novel educational game mechanics
- Designing secure portal communication
- Reducing procedural city geometry from **~780k** → **~95k triangles**
- Removing commercial systems for compliance
- Building cross-platform integration architecture across five distinct technologies
- Developing browser-native educational games with curriculum alignment
- Creating 17+ interactive quiz question prototypes with custom physics, touch handling, and animation systems
- Solving mobile drag-drop limitations not addressed by HTML5 native APIs
- Developing circular angle mathematics for clock interaction components
- Building custom floating physics engine for educational game elements

This body of work represents experimental development beyond standard software implementation and required systematic investigation under conditions of technical uncertainty.

7. Evidence Summary

Project	Technology	Evidence Location
Math Crush	Unity WebGL	Local Unity project; itch.io deployment
Racing Game	Unity WebGL	CityGen3D-Test project; profiling logs
Kangaroo Hop	Phaser.js	github.com/dannyboy166/my-kangaroo-game
Wordle	React	github.com/dannyboy166/wordle-game
2048	Vanilla JS	github.com/dannyboy166/2048-game
Memory Mania	HTML5 Canvas	github.com/dannyboy166/memory-game
Spelling Snake	HTML5	github.com/dannyboy166/spelling-snake-game
Games Portal	HTML/CSS	github.com/dannyboy166/sasco-games-portal
Blazor Integration	C#/JavaScript	BlazorUnityBridge.cs; BubbleTest project
Quiz Templates	HTML5/CSS/JS	github.com/dannyboy166/quiz-templates

Document prepared for R&D Tax Incentive purposes under Division 355 of the Income Tax Assessment Act 1997.